01 / 训练到底在做什么？

从你的采样电路，到一个能够估计误差的小函数

你已有 1 GS/s 差分晶体管级采样网络。接下来不会让 AI 修改网表，而是用电路产生的数据，训练一个数字后校正函数。网络每次回答：这个采样点应该减去多少误差？

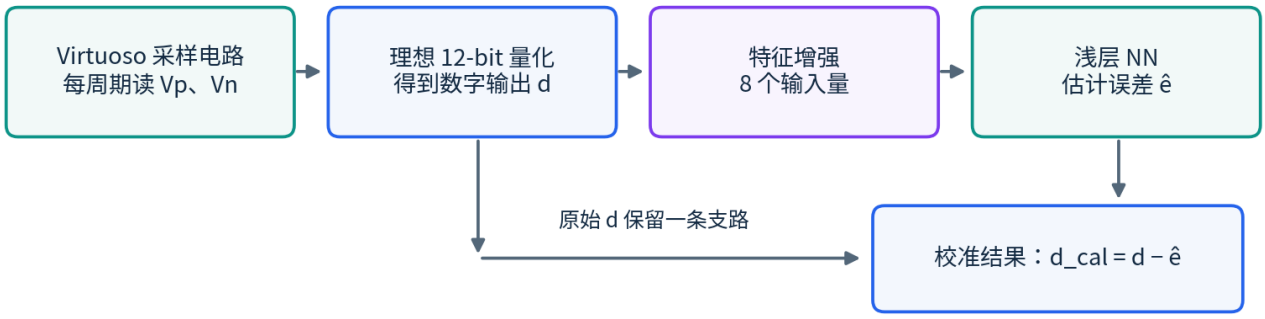


图 1：接入你现有电路的完整链路。量化步骤由 MATLAB 补上。

训练前不知道的东西，叫作“ 权重 ”

网络是一个带参数的数学函数 $\hat{e} = f(X; \theta)$。X 是当前数字输出、输出变化量及温度等输入； θ 是 39 个可以调整的数值。训练的工作就是用数据找到这些数值。训练完成后固定它们。

词语	在你的实验中是什么意思
输入 / 特征 X	网络用来判断误差的 8 个量
标签 e_target	训练时提供的“ 应该预测多少误差 ”
权重 / 参数 θ	函数里的系数，通过训练调整
推理 / 校准	权重固定后，给新数据计算误差并减掉

一个示意采样点：d=0.420，参考值为 0.415，所以标签误差为 0.005。若网络预测 $\hat{e}$ =0.004，校准结果就是 0.416，尚余 0.001 的误差。这里数值均为归一化单位，不是伏特。

代码和本文说明是依据论文方法写的独立实现，不是作者发布的程序。演示结果来自合成 RC 数据；尚未用你的真实电路数据训练。

02 / 先准备能训练的数据

一条 Virtuoso 波形，变成很多组 “ 输入 + 标签 ”

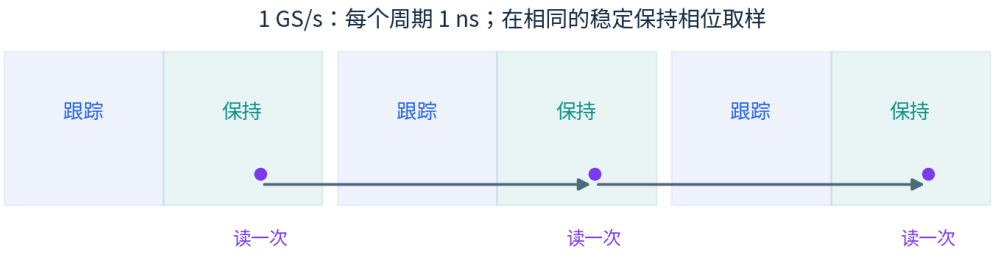


图 2：每周期只读取一对稳定保持电压。具体时钟相位按你的电路确定。

导出 time_s、vp_v、vn_v，先去掉启动过程。差分保持电压为 vp-vn。每条记录只包含一个输入频率、幅度和温度，建议取得 8192 个有效样本；1 GS/s 下相当于 8.192 μ s 的样本窗口。

为什么还要理想量化？

采样网络输出仍是模拟电压。论文的网络处理 ADC 数字输出，所以先加 12-bit 理想量化。如果差分范围是 -0.5 至 +0.5 V，步长为 1 V/4096≈244 μ V。这只是占位范围，必须换成你的电路范围。

数据	应当是什么
time_s	每周期一个样本，间隔 1 ns；单位秒
vp_v、vn_v	同一时刻的正、负保持电压；单位伏特
fin_Hz	实际施加的单音频率；单位 Hz
temperature_C	该次仿真的温度；单位 ° C

程序把量化后的等效电压按固定满量程缩放为 d，通常在 -1 到 +1 之间。这样高次幂和 tanh 输入的数值更容易处理。不能为每条波形各自缩放满量程，否则网络会失去幅度差异。

```
r = adc_nn_record(tHeld, vpHeld, vnHeld, 25, fin, cfg, 'train_01');
```

如果你已经仿真了后续 ADC 并有数字码，先按其转换关系还原等效电压，设置 quantize=false，避免重复量化。

03 / 为什么要给网络 8 个输入？

把电路知识提前告诉网络，减少需要学习的东西

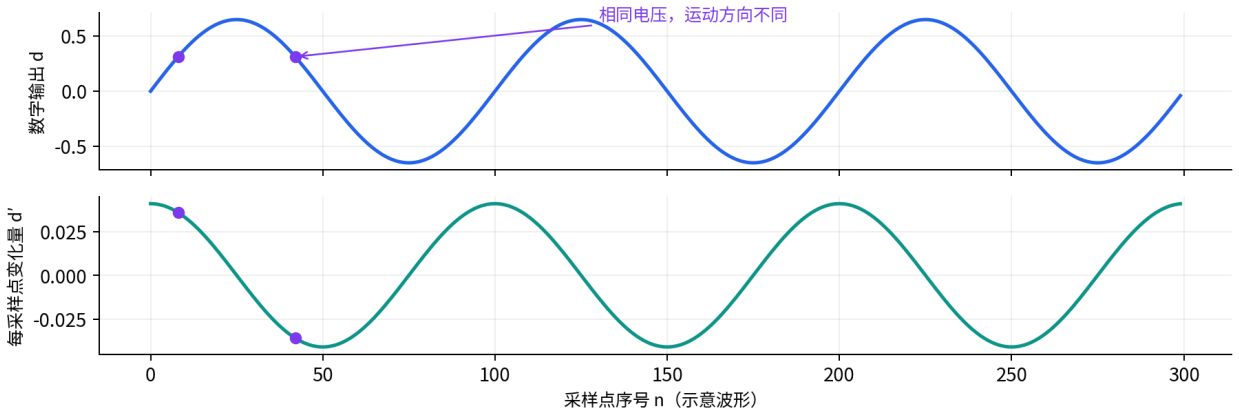


图 3：同一电压可能处于上升或下降过程；采样动态误差可能不同。

只给 d，网络无法区分“正在上升的这个电压”和“正在下降的这个电压”。采样开关电阻与电容共同产生动态误差，因此还给它 d′，即输出在相邻采样时刻之间变化有多快。

论文的特征增强

8 维输入是 $[d, d', d', d, d'\,d^2, d'\,d^3, d'\,d^4, d'\,d^5, T]$。这些乘积不是网络学习出来的，是程序按论文式（6）提前计算的，用来描述“变化速度 × 电压相关非线性”。

d′ 由 17-tap 反对称 FIR 求得。这里用“每采样点变化量”，不乘 10^9 得到每秒导数；之后只用训练集的均值和标准差统一标准化。验证及测试沿用同一套标准化参数。

```
[X, idx] = adc_nn_features(r, cfg);
```

这些特征只需数字输出和温度，不需要读取晶体管内部电流。若第一轮只仿真 25 °C，温度是常数，网络不能学到温度变化规律；后续增加温度数据后重新训练。

04 / 神经网络内部是什么？

3 个 tanh 单元、一条线性支路、一次加权求和

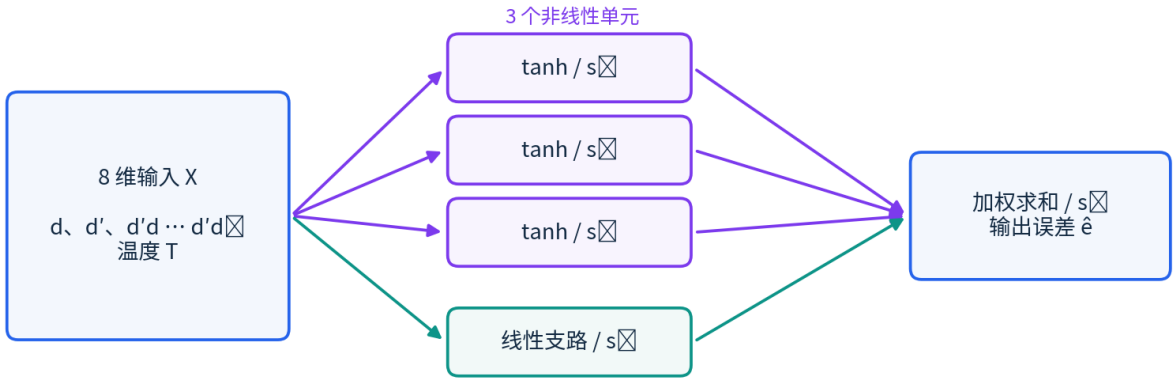


图 4：对应论文 s_1 至 s_5 的函数结构；不是 5 个隐藏层神经元。

每个非线性单元先把输入乘以权重并相加，再加一个偏置，最后经过 tanh。tanh 是一个平滑的 S 形数学函数，用于表达输入与误差之间的非线性关系。

```
z = W*X + b;
nonlinear = tanh(z);
ê = errorScale * (v'*nonlinear + a'*X + c);
```

W、b 决定 3 个非线性单元如何响应输入；v 决定它们对输出的贡献；a 是线性支路的系数；c 是输出偏置。代码把线性支路的输出增益折入 a，表达能力保持一致，共 39 个可训练参数。

“学习”就是调整这些系数

一个简化示意：某单元收到 d=0.4、d′=0.05，若系数为 1.2、-0.8，偏置为 0.1，那么内部值为 1.2×0.4-0.8×0.05+0.1=0.54，tanh 后约为 0.493。这些系数仅为解释用，真实系数由训练得到。

你无需手工算每个参数如何变化。Deep Learning Toolbox 用自动微分计算梯度，Adam 按梯度更新权重。代码未实施作者最终的对称约束和反复剪枝流程。

05 / 标签、损失与更新怎样连接？

这是训练发生的地方；校准时不再生成标签

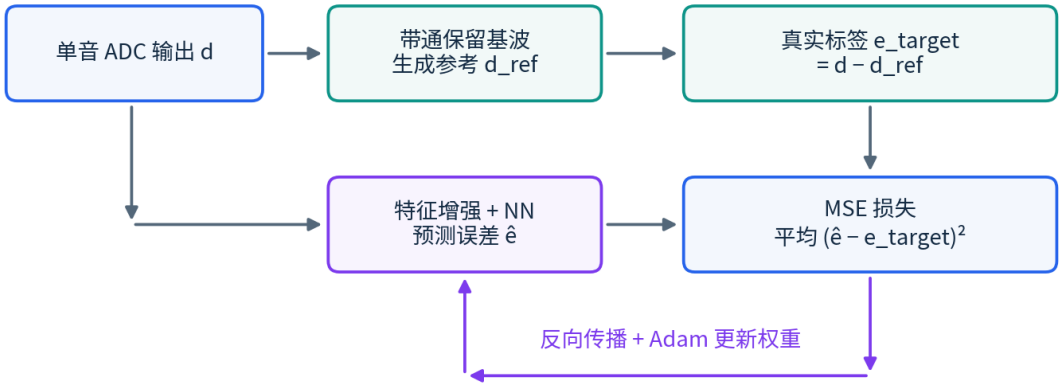


图 5：上方生成误差标签，下方预测误差，比较后更新权重。

训练只用已知频率的单音。数字输出 d 包含基波、谐波及噪声。带通滤波保留基波，得到参考 d_ref；标签为 e_target=d-d_ref。网络预测 ê，损失为平均 (ê -e_target) ² 。

带通必须在基波处保持单位增益，并对齐时间。否则错误的幅度或延迟会被当成 ADC 失真。本实现使用中心卷积与边界裁剪处理延迟，不让滤波开头和结尾的伪差参与训练。

一次训练循环

① 取一批输入/标签；② 计算预测误差；③ 计算 MSE；④ 自动微分求梯度；⑤ Adam 更新参数；⑥ 换下一批数据，重复。一个 epoch 是把全部训练样本看一遍，mini-batch 是一次用于更新的一小批样本，默认最多 2048 个。

```
[loss, grad] = dlfeval(@gradients, ...);
[p, avg, avgSq] = adamupdate(p, grad, ...);
```

这里的参考只保留实际输出的基波，并不等于无误差的真实输入。这个目标主要用于抑制谐波，不能据此恢复绝对增益/相位或把随机噪声全部消除。滤波器只在训练阶段使用。

06 / 怎样知道训练真的有效？

训练集用于学习，验证集用于选择，测试集用于独立评估

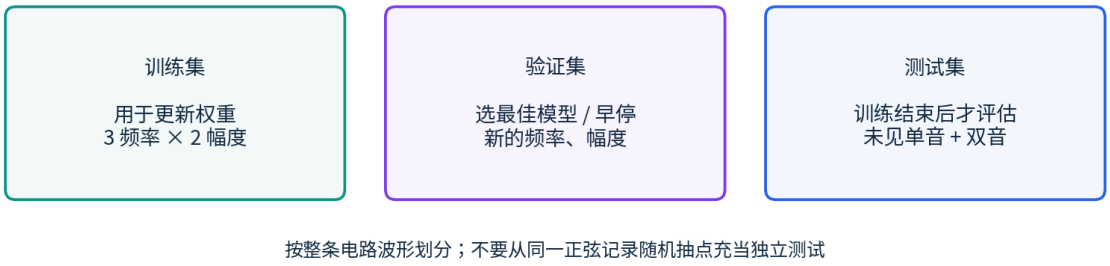


图 6：按整条波形或输入条件划分三组数据。

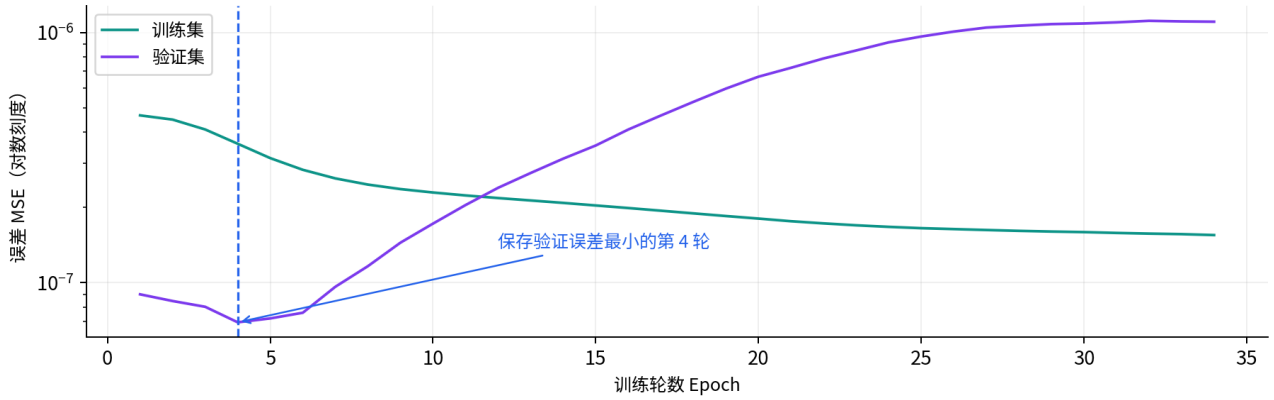


图 7：NumPy 参考实现的合成 RC 数据检查曲线；不是 MATLAB 原生运行或你的电路结果。

训练误差低，只说明网络适应了已看过的数据。验证集不更新权重，用来选择模型。程序保留验证 MSE 最小的一轮，连续 30 轮没有改善就早停；这张检查图中最佳是第 4 轮，不是最后一轮。

最终测试再换一组没有参与模型选择的频率/幅度。还要用单音训练的模型测试双音，观察互调分量。第一次实验可以固定温度，成功后再加入温度扫描。

两条曲线不一定平滑，也不必一直下降。以独立数据上的 FFT、SNDR、SFDR、基波幅度和噪声底作为最终判断。训练 MSE 与 SNDR 不是同一个指标。

07 / 训练结束后，怎样校准新数据？

权重冻结，输入新波形，直接减去预测误差

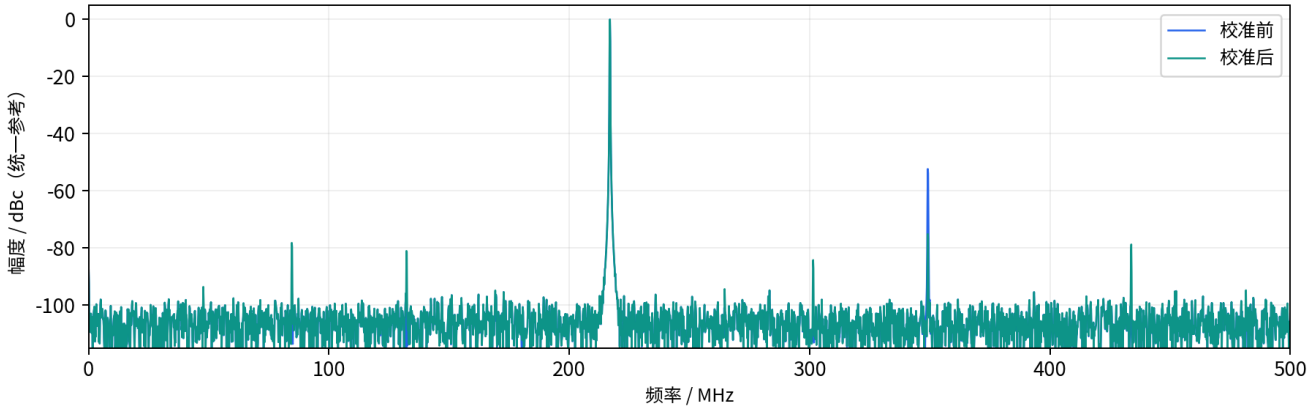


图 8：独立合成 RC 单音测试的频谱示例；不能替代你的 Virtuoso 实验。

推理只做：新数字输出 → 相同特征/标准化 → 已训练的固定函数 → ê → d_cal=d- ê 。它不再用基波滤波标签，不看理想输入，也不用输入频率来计算校正。fin 仅供评估函数识别基波。

```
[model, history] = adc_nn_train(train, validation, cfg);
out = adc_nn_predict(model, test);
report = adc_nn_evaluate(model, test, true);
save('my_adc_calibrator.mat', 'model', 'history');
```

out.dCal 是校准后的归一化数字信号；out.vCal 是差分等效电压；out.correctedCode 可以有小数。不要立刻重新舍入到 12 bit，否则会损失细小的数字校正。

必须检查的现象

谐波/互调是否下降；未见过的输入是否仍有效；基波是否被明显削弱；有没有新杂散；低失真条件是否被损害。一个已经接近理想量化噪声的电路，NN 的 SNDR 改善可以很小。

报告应区分：原论文的实测结果、代码的合成演示结果、你真实晶体管级电路的数据结果。本文尚未把后两者混为一谈。

08 / 你现在按什么顺序操作？

先跑通演示，再跑真实波形，最后扩展实验

顺序	动作	对应文件
1	解压，运行合成演示	demo_adc_nn.m
2	确认差分满量程、位数	adc_nn_config.m
3	每周期读取稳定保持电压	adc_nn_sample_waveform.m
4	准备 6 训练 + 1 验证 + 1 测试记录	manifest_template.csv
5	运行真实数据训练	run_virtuoso_data.m
6	查看 FFT 和指标，保存模型	adc_nn_evaluate.m

```
[model, history, results] = demo_adc_nn;
% ■■■■■■■■■■
[model, history, results] = run_virtuoso_data('manifest.csv');
```

1 GS/s、8192 点的起始建议：训练频率约 85.571、150.269、244.507 MHz，各两个幅度；验证约 195.435 MHz；测试约 216.919 MHz。精确相干频率与 CSV 示例见 README。

常见问题，从数据开始排查

现象	优先检查
量化溢出	vmin/vmax 是否是差分满量程；输入峰值是否过大
训练不收敛	取样相位、启动数据、单位、标签延迟、标准化
训练有效，测试变差	是否只训练一个条件；是否测试超出训练范围
几乎没有性能改善	电路是否已很线性；是否已由量化/噪声限制
标签拒绝某频率	高次谐波可能混叠到基波；换频率或改滤波器

当前检查：14 个 .m 文件通过语法解析；对应数学流程已在 NumPy 检查。环境没有 MATLAB，所以未实际执行 Deep Learning Toolbox；先在你的 MATLAB 运行 demo，遇到报错把完整信息发来。详见 VALIDATION.md。

本文依据：Molendijk 等，TCAS-I 2026，式（6）、Fig. 3/4 与 Section III；MATLAB 自动微分与 Adam 接口依据 MathWorks 官方文档。代码 README 给出了链接和所有实现差异。

09 / 按你的电路参数开始

0.6 Vpp 输入、1.2 Vpp 输出、0.5 V 共模、496.09375 MHz

前面几页用通用配置解释方法。这一页对应你刚给出的实际参数；run_virtuoso_data 已改用 adc_nn_user_config。温度先取 25 ° C，保持 12-bit 理想量化。

参数	实际配置
输入差分满量程	0.6 Vpp：若以 0 V 为中心，输入范围为 ±0.3 V
输出差分满量程	1.2 Vpp：量化 Vp-Vn，范围为 ±0.6 V
共模电压	0.5 V；不要作为差分范围的中心
采样率 / 输入频率	1 GHz / 127×1 GHz÷256 = 496.09375 MHz
12-bit 输出量化步长	1.2 V÷4096 ≈ 292.969 μV

接近 Nyquist，需要改变滤波器

Nyquist 是 500 MHz，你的输入只比它低 3.90625 MHz。通用 17-tap 导数滤波器在此点幅度约为理想值的 6.18%，因此用户配置改为 513 taps，数值检查幅度误差约 0.044%。标签带通半宽缩到 0.75 MHz，以免跨过边界。

这是一项近 Nyquist 算法适配，不能声称复现了论文 17-tap 的面积、功耗或延迟。此滤波器的因果延迟是 256 个周期，离线训练用时间对齐处理。真实电路是否能有效校准仍要测试。

第一轮：固定频率、固定温度、改变幅度

数据集	满量程比例	差分输入 Vpp
训练	45%、65%、85%	0.27、0.39、0.51 V
验证	70%	0.42 V
测试	80%	0.48 V

```
[model, history, results] = ...
run_virtuoso_data('manifest_user_496MHz.csv');
```

先按 manifest 准备五份独立电路波形，各取 8192 个有效保持样本。这一轮只验证幅度泛化；若要验证频率、温度或双音，必须补相应数据。输入输出的两倍增益也不会由基波标签自动消除。